

AI-Powered Transaction Processing Pipeline

Homework 6 Capstone

Created by Oleksandr Mochalov

The Problem

- Raw banking transactions need to be validated, screened for fraud, checked for compliance, and settled
- Built with a 4-agent AI workflow: **Specification → Code Generation → Unit Tests → Documentation**
- Language-agnostic assignment — implemented in **Python**

Architecture



Stage 1 — Validation

- Required fields, `Decimal`-parsed amount, ISO 4217 currency, ISO-8601 timestamp, allowed transaction type
- First failing check wins — one clear rejection reason
- Resolves the sample data's ambiguous negative-amount case: **amounts must always be positive**, regardless of `transaction_type`

Stage 2 — Fraud Detection

- Risk score built from: high-value (+40), very-high-value (+20), cross-border (+20), unusual timing 00:00–06:00 UTC (+20), wire transfer (+10)
- `flagged_for_review` when amount > \$10,000 **or** score $\geq$ 40
- `risk_level` : low / medium / high

Stage 3 — Compliance Check

- Restricted-country hold (synthetic test coverage — no sample data hits this)
- **Cross-border + unusual timing together** → held for review, even at low dollar amounts
- Amount > \$10,000 alone → CTR-flagged (still settles)

Stage 4 — Settlement

- Clear / CTR-flagged → assigned a `settlement_id` and `settled_at` timestamp
- Held-for-review → passes through untouched, no settlement

Results on the 8 Sample Transactions

Outcome	Count	Transactions
Settled	5	TXN001, TXN002, TXN003, TXN005, TXN008
Held for review	1	TXN004 (cross-border + 02:47 UTC)
Rejected	2	TXN006 (invalid currency XYZ), TXN007 (negative amount)

Skills, Hooks & MCP

- **Skills:** `/write-spec`, `/run-pipeline`, `/validate-transactions`
- **Hook:** `PreToolUse` on `Bash` — blocks `git push` if `pytest --cov=pipeline --cov-fail-under=80` fails
- **MCP:** `context7` (library research) + custom `pipeline-status` FastMCP server (`get_transaction_status`, `list_pipeline_results`, `pipeline://summary`)

Test Coverage

- 84 tests across validator, fraud detector, compliance checker, settlement processor, common utilities, currency codes, orchestrator integration, an in-memory pipeline-chaining integration test, and the MCP server
- **100% coverage on `pipeline/`** — comfortably above the 80% hard gate and the 90% target
- Isolated from the real `shared/` directory via pytest's `tmp_path`

Lessons Learned

- Writing the specification *before* code made the fraud/compliance decision rules unambiguous and directly testable
- Keeping stage functions pure (no file I/O) made unit testing trivial and kept the orchestrator as the single owner of the `shared/` protocol
- PII-safe logging (masked accounts, dropped descriptions) needed to be a shared helper, not per-stage logic, to stay consistent
- Claude Code hooks and MCP servers only reload at session start — plan the build order around that

Thank You

Created by **Oleksandr Mochalov**